

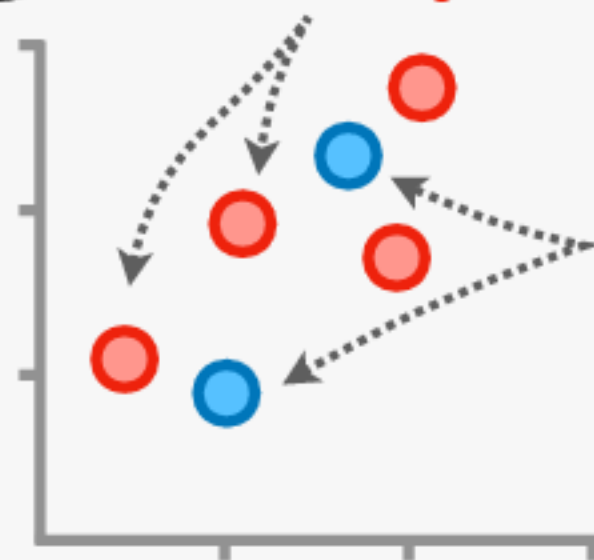
Chapter 02

Cross Validation!!!

Cross Validation: Main Ideas

1

The Problem: So far, we've simply been told which points are the **Training Data**...



...and which points are the **Testing Data**.

However, usually no one tells us what is for **Training** and what is for **Testing**.

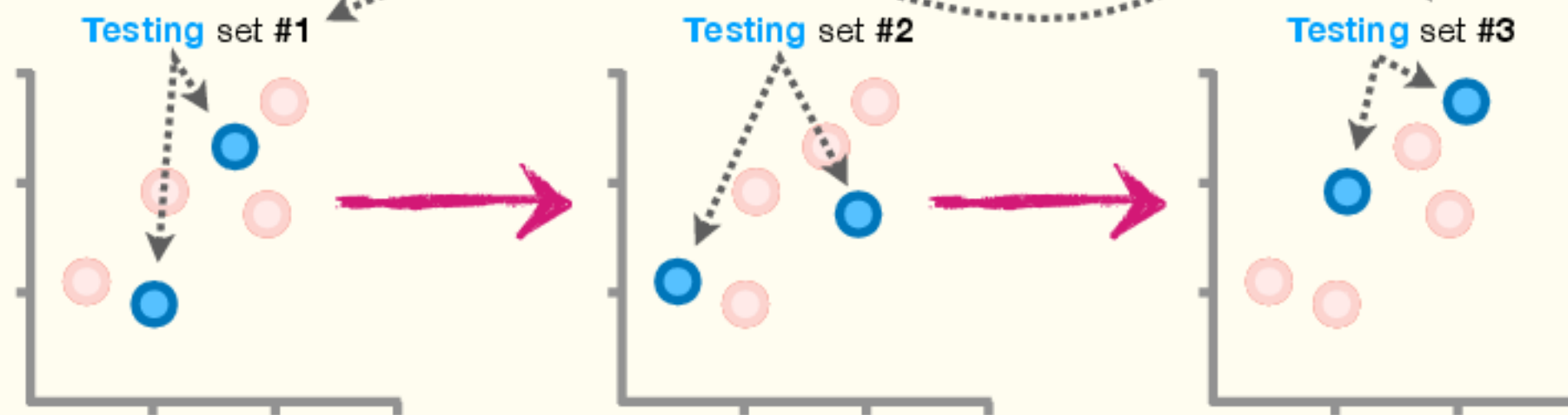


How do we pick the best points for **Training** and the best points for **Testing**?

2

A Solution: When we're not told which data should be used for **Training** and for **Testing**, we can use **Cross Validation** to figure out which is which in an unbiased way.

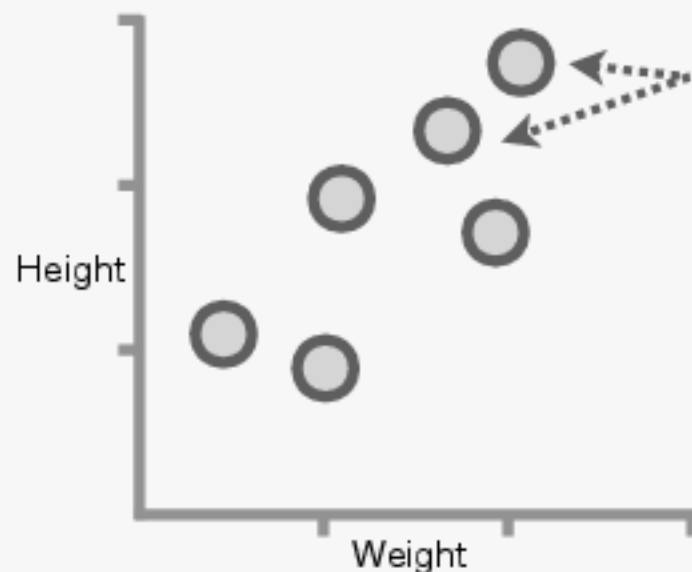
Rather than worry too much about which specific points are best for **Training** and best for **Testing**, **Cross Validation** uses *all* points for both in an *iterative* way, meaning that we use them in steps.



BAM!!!

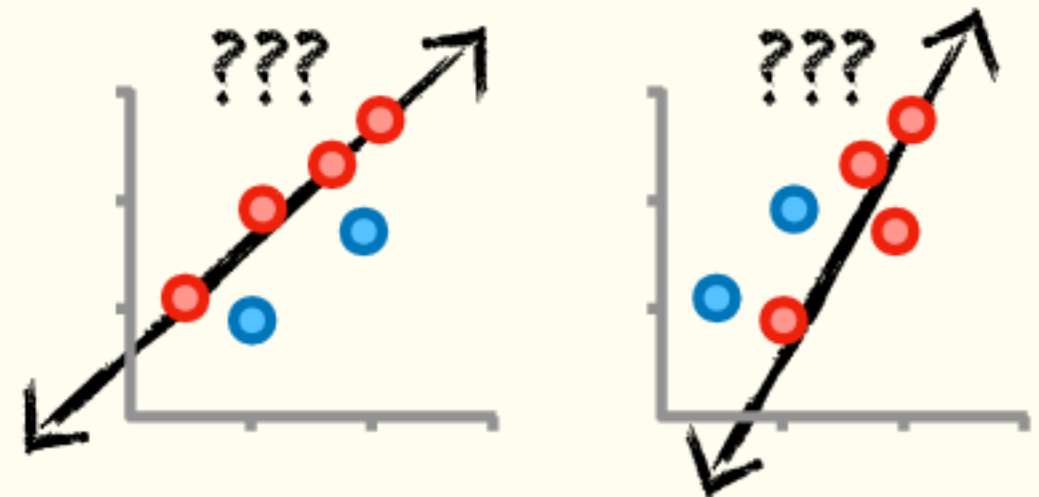
Cross Validation: Details Part 1

- ① Imagine we gathered these 6 pairs of Weight and Height measurements...

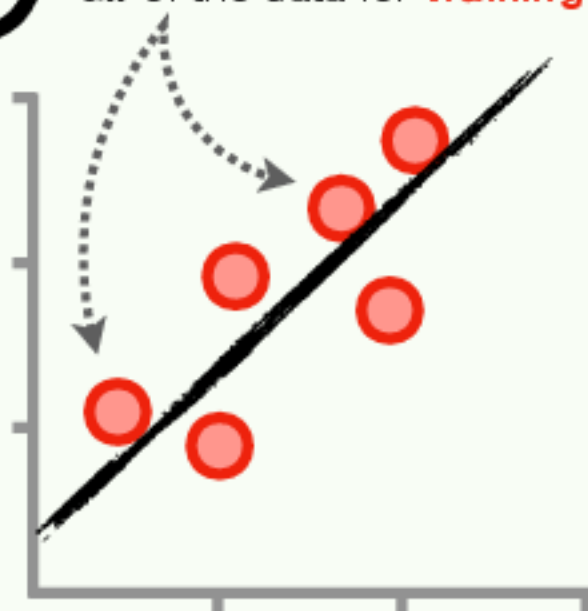


...and because we see a trend that people with larger Weights tend to be taller, we want to use Weight to predict Height...

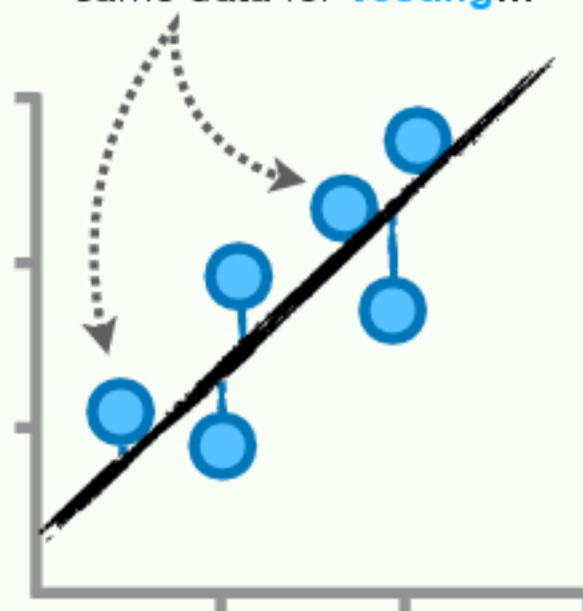
- ② ...so we decide to fit a **line** to the data with **Linear Regression** (for details, see **Chapter 4**). However, we don't know which points to use for **Training** and which to use for **Testing**.



- ③ A terrible idea would be to use **all** of the data for **Training**...



...and then reuse the exact same data for **Testing**...



...because the only way to determine if a machine learning method has been **Overfit** to the **Training Data** is to try it on new data that it hasn't seen before.

TERMINOLOGY ALERT!!!

Reusing the same data for **Training** and **Testing** is called **Data Leakage**, and it usually results in you believing the machine learning method will perform better than it does because it is **Overfit**.

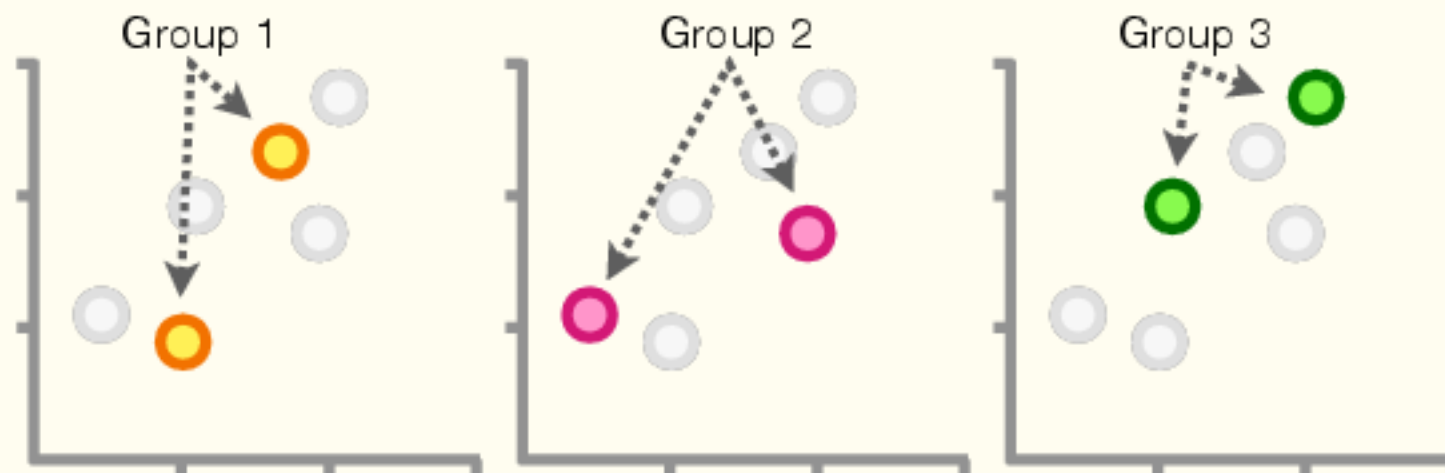
Cross Validation: Details Part 2

- ④ A slightly better idea is to randomly select some data to use *only* for **Testing** and select and use the rest for **Training**.

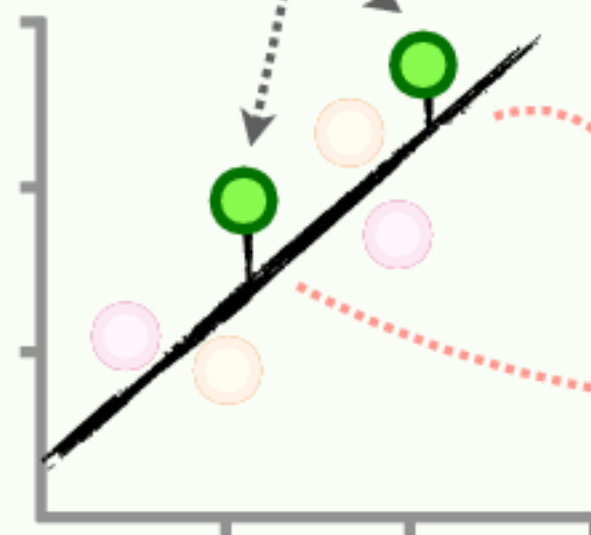
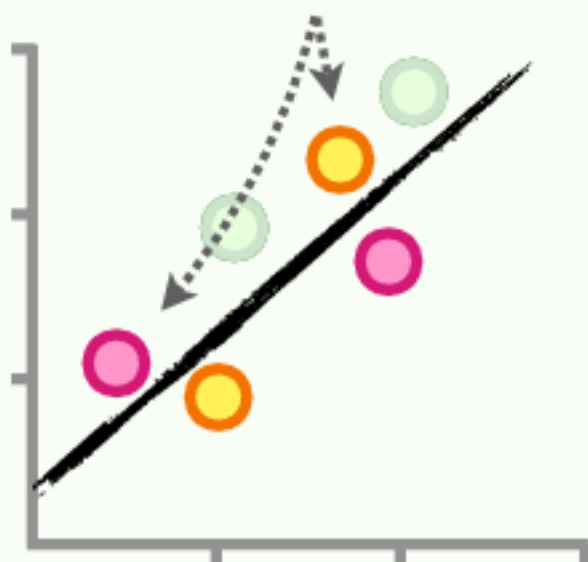


This avoids **Data Leakage**, but how do we know we selected the best data for **Testing**?

- ⑤ **Cross Validation** solves the problem of not knowing which points are the best for **Testing** by using them *all* in an *iterative* way. The first step is to randomly assign the data to different groups. In this example, we divide the data into **3** groups, where each group consists of **2** points.



- ⑥ Now, in the first **Cross Validation** iteration, we'll use **Groups 1** and **2** for **Training**... ...and **Group 3** for **Testing**.



- ⑦ Then, just like before, we can measure the errors for each point in the **Testing Data**...

Iteration #1:
**Black Line
Errors**

I
I

...however, unlike before, we don't stop here. Instead, we keep iterating so that **Groups 1** and **2** can each get a turn to be used for **Testing**.

Cross Validation: Details Part 3

- 8 Because we have 3 groups of data points, we'll do 3 iterations, which ensures that each group is used for **Testing**. The number of iterations are also called **Folds**, so this is called **3-Fold Cross Validation**.

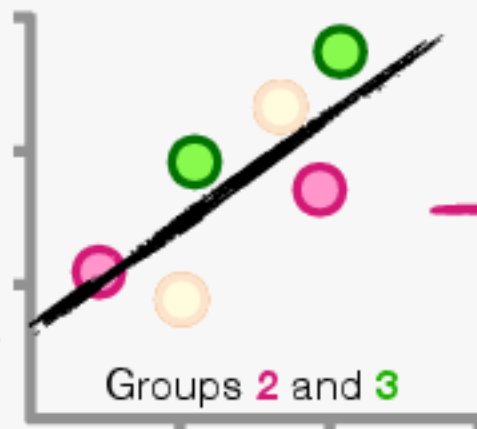
Gentle Reminder:
These are the original 3 groups.



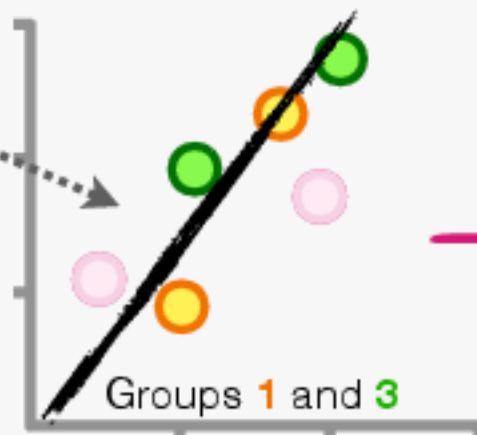
- 9 So, these are the 3 iterations of **Training**...

NOTE: Because each iteration uses a different combination of data for **Training**, each iteration results in a slightly different fitted **line**.

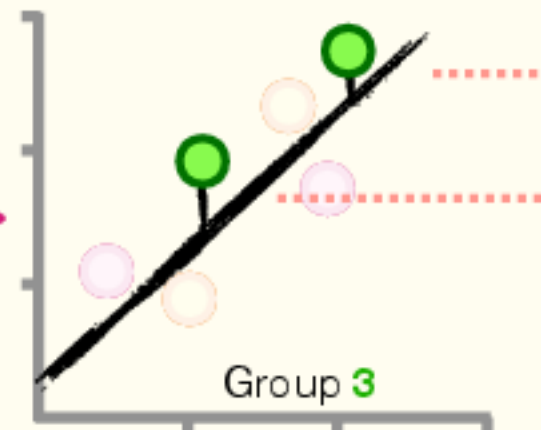
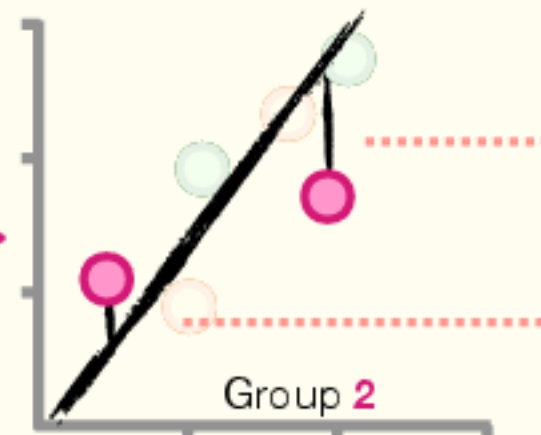
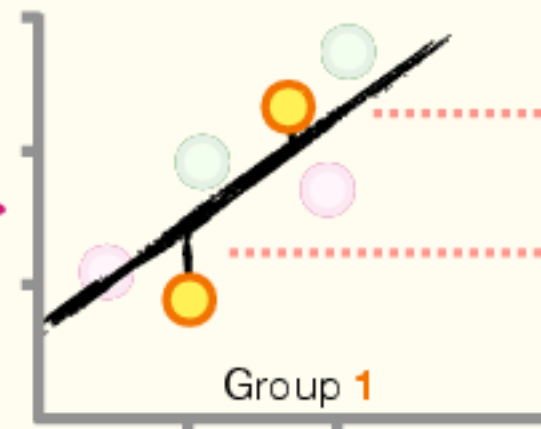
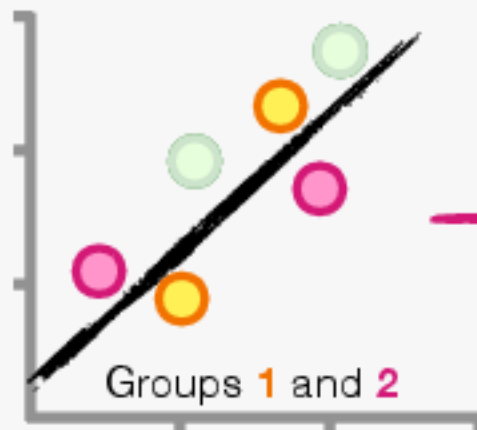
Iteration #1



Iteration #2



Iteration #3



- 10 ...and these are the 3 iterations of **Testing**.

A different fitted line combined with using different data for **Testing** results in each iteration giving us different prediction errors.

We can average these errors to get a general sense of how well this model will perform with future data...

...or we can compare these errors to errors made by another method.

Cross Validation: Details Part 4

11 For example, we could use **3-Fold Cross Validation** to compare the errors from the **black line** to the errors from the **green squiggle**.

Gentle Reminder:
These are the original 3 groups.

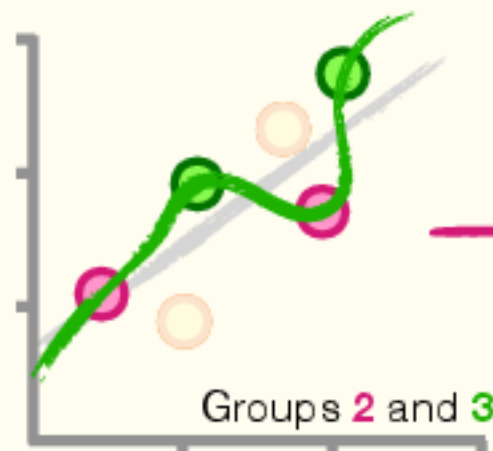


12 Training

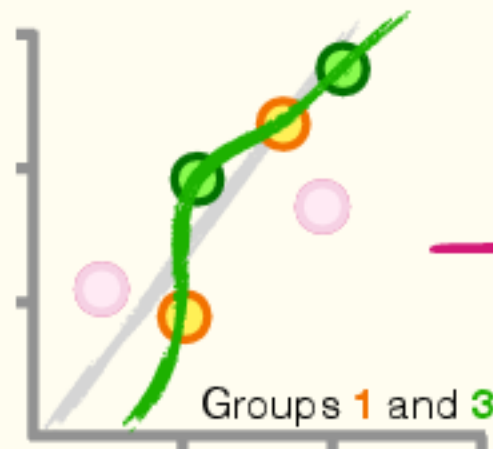
Again, because each iteration uses a different combination of data for **Training**...

...each iteration results in a slightly different fitted **line** and fitted **squiggle**.

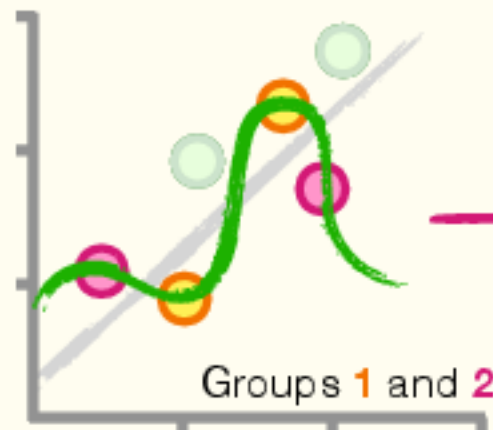
Iteration #1



Iteration #2



Iteration #3



13 Testing

Total Green Squiggle Error

Total Black Line Error

vs.

vs.

vs.

In this case, all 3 iterations of the **3-Fold Cross Validation** show that the **black line** does a better job making predictions than the **green squiggle**.

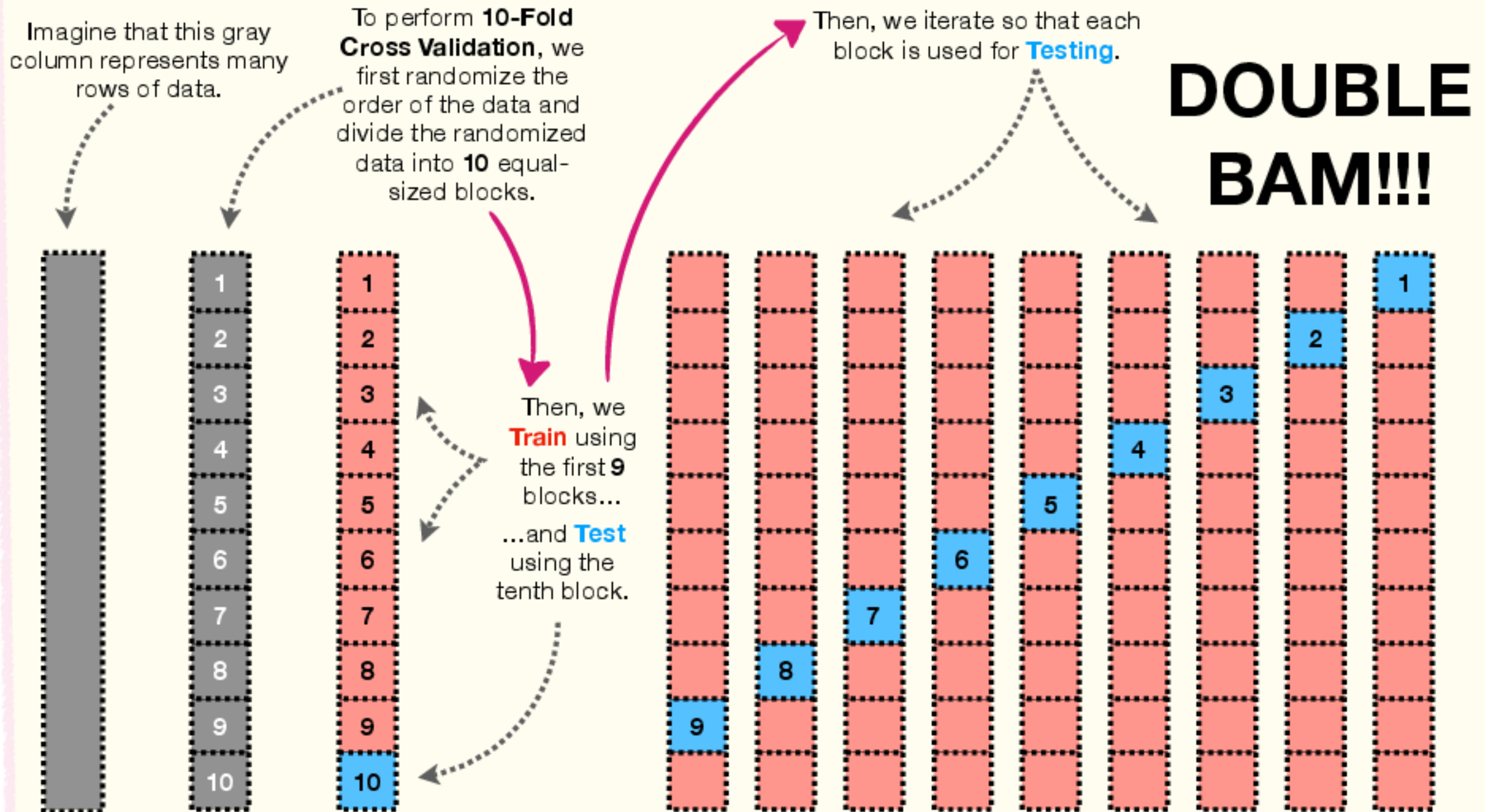
By using **Cross Validation**, we can be more confident that the **black line** will perform better with new data without having to worry about whether or not we selected the best data for **Training** and the best data for **Testing**.

BAM!!!

NOTE: In this example, the **black line** consistently performed better than the **green squiggle**, but this isn't usually the case. We'll talk more about this later.

Cross Validation: Details Part 5

14 When we have a lot of data, **10-Fold Cross Validation** is commonly used.



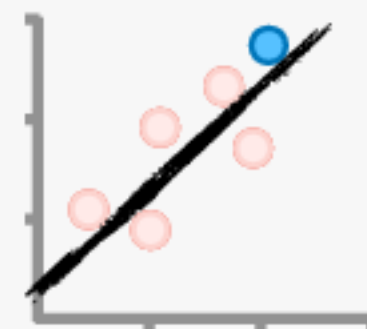
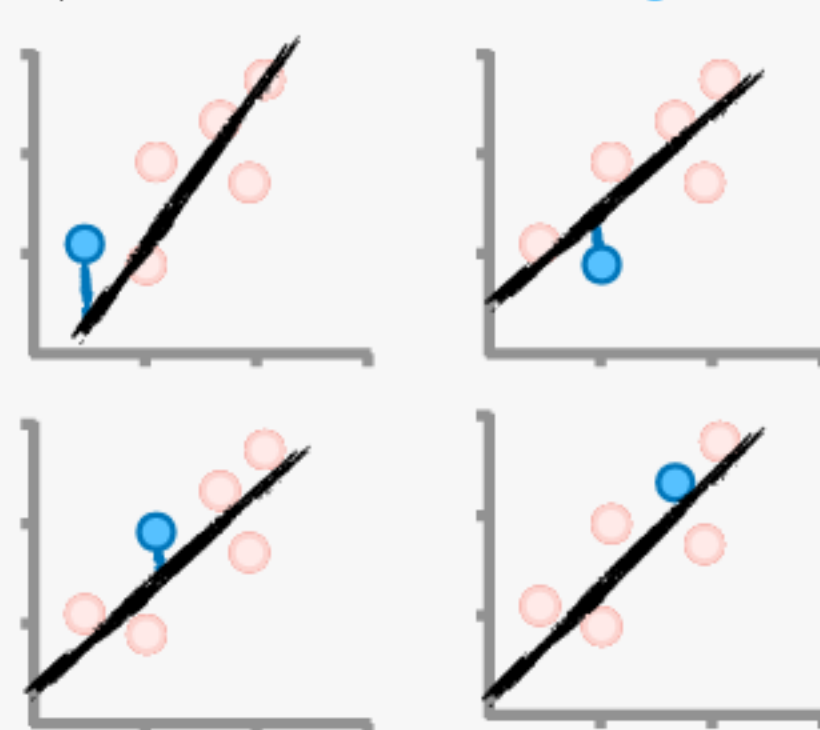
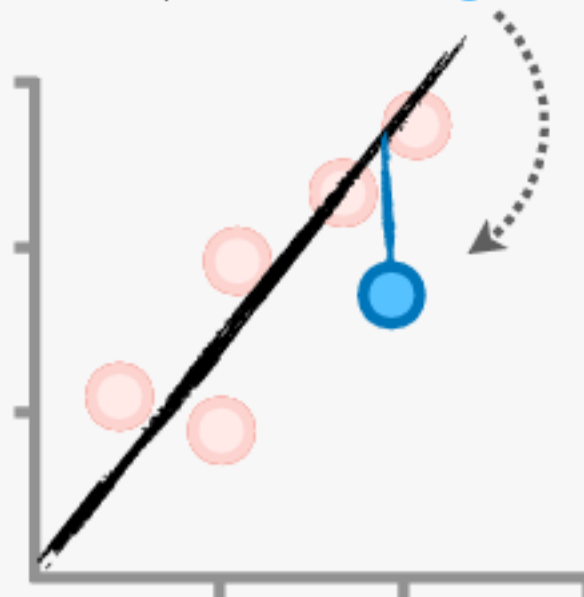
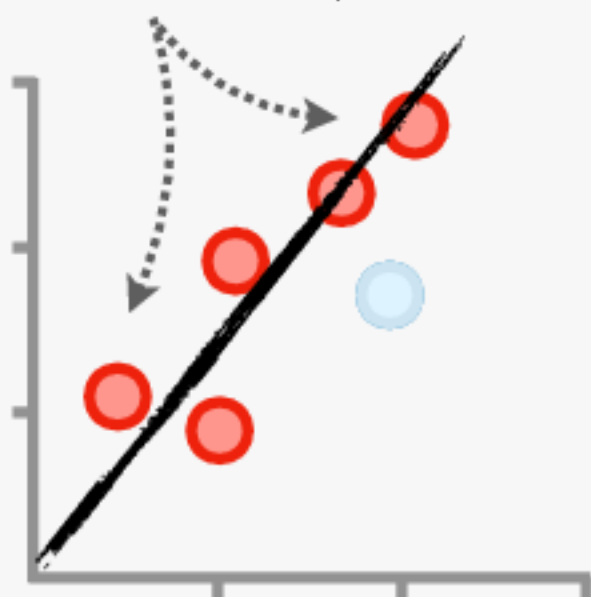
Cross Validation: Details Part 6

15 Another commonly used form of **Cross Validation** is called **Leave-One-Out**.

Leave-One-Out Cross Validation uses all but one point for **Training**...

...and uses the one remaining point for **Testing**...

...and then iterates until every single point has been used for **Testing**.

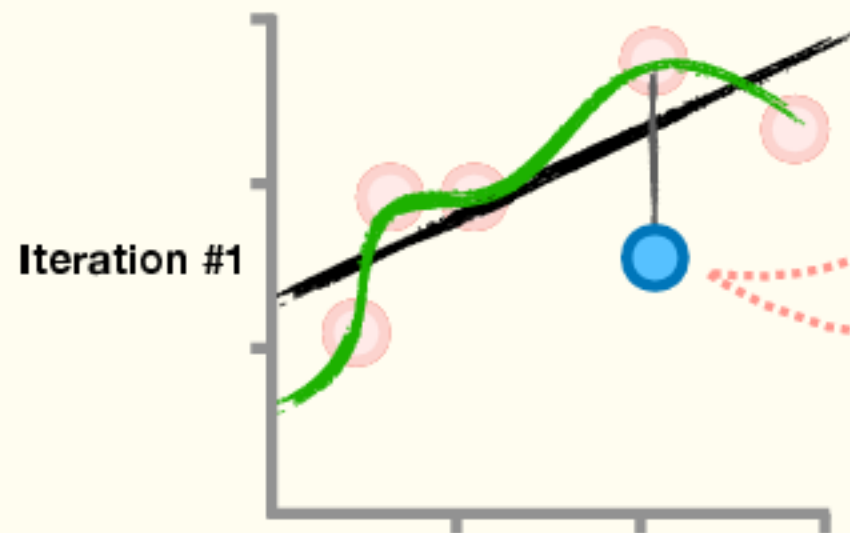


Hey **Norm**, how do I decide if I should use **10-Fold** or **Leave-One-Out Cross Validation**?

Some experts say that when the dataset is large, use **10-Fold Cross Validation**, and when the dataset is very small, use **Leave-One-Out**.

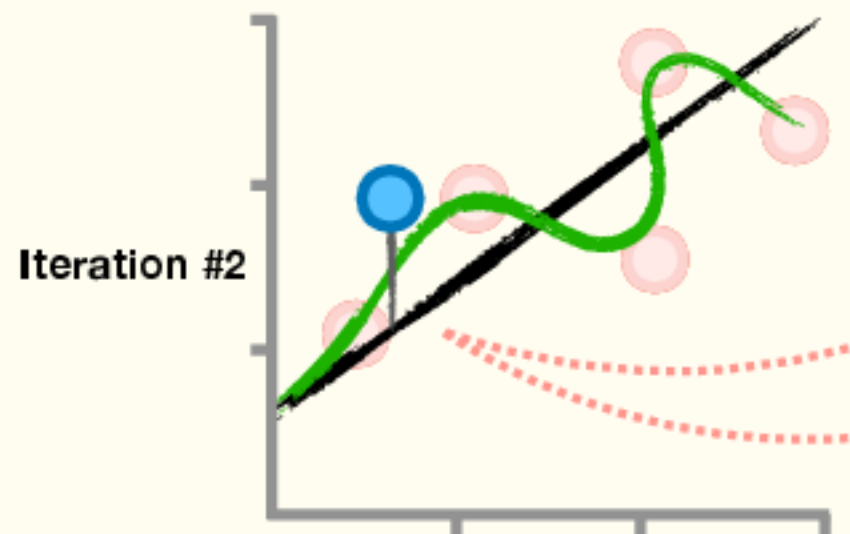
Cross Validation: Details Part 7

16 When we use **Cross Validation** to compare machine learning methods, for example, if we wanted to compare a **black line** to a **green squiggle**...



...sometimes the **black line** will perform *better* than the **green squiggle**...

vs.



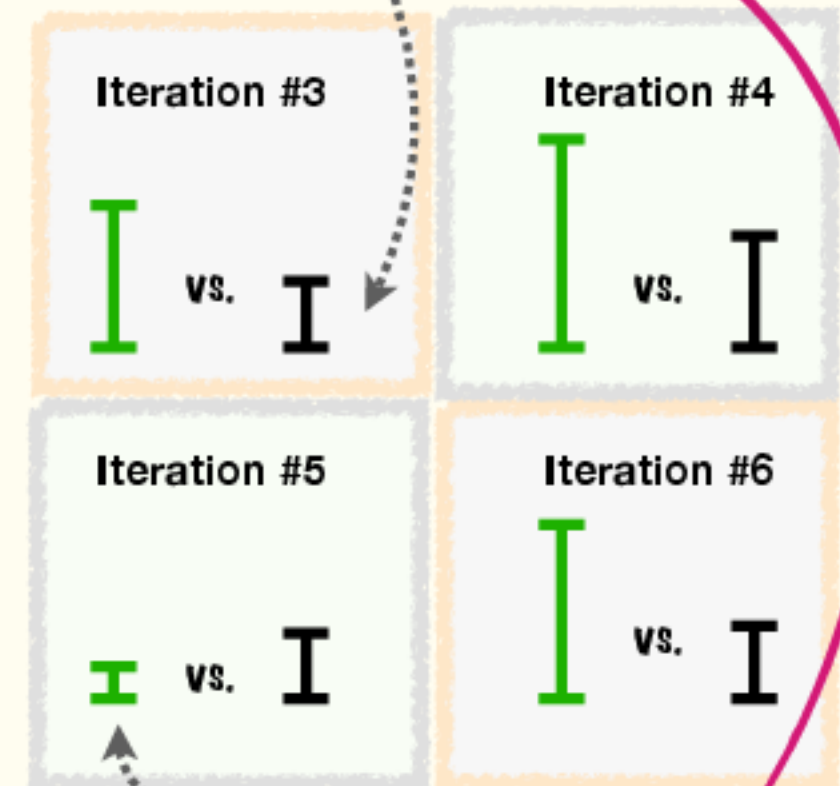
...and sometimes the **black line** will perform *worse* than the **green squiggle**.



vs.



And, after doing all of the iterations, we're left with a variety of results, some showing that the **black line** is better...



...and some showing that the **green squiggle** is better.

When the results are mixed, how do we decide which method, if any, is better? Well, one way to answer that question is to use **Statistics**, and that's what we'll talk about in the next chapter.

TRIPLE BAM!!!